IDS 705 — Machine Learning

Study Guide

Topics Covered

1. Probability & Estimation (Bayes, MLE, MAP)
2. Bias-Variance Tradeoff
3. K-Nearest Neighbors (KNN)
4. Linear Regression
5. Logistic Regression & Perceptron
6. Performance Evaluation (Metrics, ROC, PR Curves)
7. Model Selection & Cross-Validation
8. Decision Theory
9. Regularization (Ridge, LASSO, Elastic Net)
10. Generative vs. Discriminative Models (Naive Bayes, LDA, QDA)
11. Trees & Ensembles (Decision Trees, Random Forests, Boosting)
12. Neural Networks & Deep Learning
13. CNNs & Transformers
14. PCA & Dimensionality Reduction
15. Clustering (K-Means, GMM, DBSCAN, Hierarchical)
16. Reinforcement Learning
17. Gradient Descent & Optimization
18. Practical Reference & Exam Quick Tips

1. Probability & Estimation

Why Probability in ML?

Machine learning models are fundamentally about making predictions under uncertainty. Probability gives us a rigorous language to express what we know, what we don't know, and how confident we are. Almost every ML model either implicitly or explicitly uses probabilistic reasoning.

Bayes' Theorem — Updating Beliefs

Bayes' theorem tells us how to update our belief about something (a hypothesis, a parameter, a class label) after seeing new evidence.

$$P(Y \mid X) = P(X \mid Y) \times P(Y) \;/\; P(X)$$

- **P(Y | X)** = Posterior: our updated belief about Y after observing X
- **P(X | Y)** = Likelihood: how probable is the data X if Y were true?
- **P(Y)** = Prior: our belief about Y before seeing any data
- **P(X)** = Evidence / normalizing constant: ensures the posterior sums to 1

Real-world example: Suppose you test positive for a rare disease (1% prevalence). The test is 99% accurate. Should you be worried? Your posterior P(disease | positive test) is actually only about 50% because the prior P(disease) is so low! Bayes' theorem captures this correctly — always incorporate the prior.

> *Key insight: The denominator P(X) = Σ P(X|Yk)P(Yk) sums over ALL possible classes. This ensures the posteriors add up to 1 and is often the hardest part to compute.*

Independence vs. Uncorrelated

These are NOT the same thing. Independence is a stronger condition.

- Independent: P(X, Y) = P(X)P(Y). Knowing X tells you absolutely nothing about Y.
- Uncorrelated: E[XY] = E[X]E[Y], or equivalently Cov(X,Y) = 0. The variables have no linear relationship.
- Uncorrelated does NOT imply independent! (e.g., Y = X² — perfectly related, but uncorrelated if X is symmetric around 0)
- Independent DOES imply uncorrelated.

Maximum Likelihood Estimation (MLE)

The core idea: find the parameter θ that makes the observed data as probable as possible. You're asking: 'Given what I saw, which parameter value was most likely to have generated this data?'

> $\theta\_MLE = \text{argmax } \Sigma \log p(x_i \mid \theta)$

We use log because: (1) it converts products into sums (much easier to differentiate), and (2) log is monotone so the argmax is the same.

For Gaussian data: MLE gives $\hat{\mu} = \bar{x}$ (sample mean) and $\sigma^2 = (1/n)\Sigma(x_i - \bar{x})^2$ (slightly biased — divides by n not n-1).

Conceptually, MLE treats the parameters as unknown fixed quantities and asks which setting of those parameters maximizes the probability of generating exactly the data you observed.

Maximum A Posteriori (MAP)

MAP is MLE + prior knowledge. You add a term that reflects your belief about the parameters before seeing the data.

> $\theta\_MAP = \text{argmax } [\log p(\text{data} \mid \theta) + \log p(\theta)]$

The critical connection to regularization:

- Ridge regression (L2 penalty) ≡ MAP estimation with a Gaussian prior on β
- LASSO (L1 penalty) ≡ MAP estimation with a Laplace prior on β

This means regularization isn't just a computational trick — it has deep probabilistic meaning. You're encoding a belief that parameters should be 'small' (Gaussian) or 'sparse' (Laplace).

> *Exam tip: When λ → 0, MAP → MLE (prior disappears). When λ → ∞, MAP → prior mean (data doesn't matter).*

---

2. Bias-Variance Tradeoff

The Decomposition

The expected test error (MSE) can be decomposed into three distinct sources:

> $E[\text{Test MSE}] = \text{Bias}^2 + \text{Variance} + \sigma^2\varepsilon$ *(irreducible noise)*

Bias

Bias measures how far off your model's average prediction is from the truth. A biased model has made the wrong assumptions about the functional form — it's systematically wrong. Think of an archer who always hits 3 inches to the left. The problem isn't randomness; it's a systematic error.

Too simple a model (underfitting) → high bias. A line trying to fit a parabola will always be wrong — no matter how much data you give it.

Variance

Variance measures how much your model's predictions change when you train it on different data samples. A high-variance model is very sensitive to which particular training set you happened to get. The archer who hits all over the place — sometimes left, sometimes right — has high variance.

Too complex a model (overfitting) → high variance. A degree-15 polynomial fits your training set perfectly but wiggles wildly on new data.

Irreducible Noise

Even with the perfect model, there's noise in the data itself — measurement error, unknown variables, inherent randomness. This $\sigma^2\varepsilon$ cannot be removed by any learning algorithm.

The Tradeoff in Practice

Increasing model complexity: bias decreases (model can capture more patterns), variance increases (more sensitive to training data). The test error traces a U-shape. The optimal complexity sits at the bottom of that U.

Scenario	Bias	Variance	Fix
Model is too simple (e.g., linear fit on curved data)	High	Low	More complex model, add features
Model is too complex (e.g., deep tree, tiny k in KNN)	Low	High	Regularize, get more data, simplify
Model is just right	Moderate	Moderate	Tune hyperparameters
Noisy dataset	—	—	Irreducible — can't fix

> *Regularization always trades bias for variance: adding a penalty increases bias but decreases variance. This is often a good deal because variance reduction is often larger than bias increase.*

---

3. K-Nearest Neighbors (KNN)

The Core Idea

KNN is gloriously simple: to predict for a new point, find its k nearest neighbors in the training data, then average their values (regression) or take a majority vote (classification). There is no explicit 'training' phase — KNN is lazy learning, storing all the data and doing all computation at prediction time.

Analogy: You're new in town and ask 'Where should I eat?' KNN answers by finding your 5 nearest neighbors and seeing where they go most. The quality of this advice depends entirely on whether your neighbors have similar tastes (similar features).

Bias-Variance and the Choice of k

k is the most critical hyperparameter in KNN, and its effect is the opposite of what you might expect:

- Small k (e.g., k=1): each prediction uses only the single closest point. The model perfectly fits every training point (0 training error!) but is extremely wiggly. Any noise in a training point directly distorts nearby predictions. Low bias, high variance.
- Large k: averaging over many neighbors smooths out noise. But if those neighbors come from different underlying classes/regions, the average is wrong. High bias, low variance.

> *The famous case: k=1 perfectly memorizes the training data (0% training error) but usually generalizes poorly. Always use CV to select k.*

Feature Scaling is Critical

KNN uses distances. If one feature ranges from 0–10,000 (e.g., income in dollars) and another from 0–1 (e.g., age in decades), the large-scale feature dominates all distance calculations. Standardize ALL features (subtract mean, divide by std) before using KNN.

When to Use KNN

- Nonlinear, complex decision boundaries with no obvious parametric form
- Small to medium datasets (scales poorly — O(nd) per prediction, O(nd) storage)
- When interpretability of individual predictions matters ('these 5 training points most influenced this prediction')
- Quick nonparametric baseline

Limitations

- Curse of dimensionality: in high dimensions, all points become roughly equidistant. 'Nearest' neighbors aren't really close. KNN degrades badly as d grows.
- Slow prediction: must compute distances to all training points for each query
- Memory intensive: stores entire training set
- Sensitive to irrelevant features

---

4. Linear Regression

The Model

Linear regression assumes the response y is a linear combination of the features plus noise: $y = X\beta + \varepsilon$. The goal is to find β that minimizes the Residual Sum of Squares (RSS) = $||y - X\beta||^2$.

OLS — Ordinary Least Squares

The closed-form solution is $\hat{\beta} = (X'X)^{-1}X'y$. This requires X'X to be invertible (no perfect collinearity between features). Geometrically, this projects y onto the column space of X — finding the closest point in the hyperplane of possible predictions.

> *Gauss-Markov theorem: OLS is the Best Linear Unbiased Estimator (BLUE) when errors have mean 0 and constant variance. 'Best' means minimum variance among all linear unbiased estimators.*

Gradient Descent Alternative

When n or p is huge, computing $(X'X)^{-1}$ is expensive ($O(p^3)$). Instead, update iteratively: $\beta \leftarrow \beta - \eta \nabla RSS$ where $\nabla RSS = 2X'(X\beta - y)$. The learning rate η controls step size — too large and you overshoot; too small and it takes forever.

R^2 — Coefficient of Determination

$R^2 = 1 - SS_{res}/SS_{tot}$. It measures the fraction of variance in y explained by the model. $R^2 = 1$ is perfect; $R^2 = 0$ means the model is no better than just predicting the mean. R^2 can be negative for terrible models on held-out test data!

Adjusted R^2 penalizes for adding more predictors (unlike regular R^2, which always increases when you add variables). Use adjusted R^2 for model comparison.

Extending to Nonlinear Relationships

Despite the name, 'linear' regression is linear in the parameters, not necessarily in the features. You can replace x with $\varphi(x) = [1, x, x^2, x^3, ...]$ and still use the same OLS formula. This is polynomial regression, and it vastly expands what shapes the model can capture.

The tradeoff: higher-degree polynomials capture more complex shapes (lower bias) but become extremely wiggly and sensitive (higher variance). A degree-10 polynomial might fit training data perfectly while being terrible on test data.

Diagnosing Problems

- High VIF (>10): multicollinearity. Two features are nearly linearly dependent. Solution: drop one, use Ridge, or PCA.
- Residuals vs. fitted plot shows a pattern: the linear assumption is violated. Add polynomial terms or transform y.
- Residuals fan out: heteroscedasticity (non-constant variance). Transform y (e.g., log) or use weighted regression.
- Outliers/leverage points: single points with extreme influence. Check Cook's distance.

---

5. Logistic Regression & Perceptron

Why Not Linear Regression for Classification?

If you use linear regression for a binary outcome (0/1), predictions can fall outside [0,1] (meaningless as probabilities) and the error structure is all wrong. Logistic regression fixes this by modeling the log-odds as linear and mapping through the sigmoid function to produce probabilities in (0,1).

The Sigmoid Function

P(Y=1|x) = σ(x'β) = 1/(1 + e^(-x'β)). This S-shaped curve:

- Outputs are always in (0,1) — valid probabilities
- The decision boundary P(Y=1|x) = 0.5 corresponds to x'β = 0 — a linear surface in feature space
- Steepness of the S-curve is controlled by the magnitude of β

Cross-Entropy Loss

Instead of RSS, logistic regression minimizes cross-entropy loss: $L = -(1/n)\Sigma[y_i \log(\hat{p}_i) + (1-y_i) \log(1-\hat{p}_i)]$. This is the log-likelihood under a Bernoulli model — maximizing it is equivalent to MLE.

> *Intuition: cross-entropy heavily penalizes confident wrong predictions. If the model says P(Y=1)=0.999 but the true label is 0, the loss term is -log(0.001) ≈ 6.9 — a very large penalty.*

Multiclass — Softmax

For K classes, replace the sigmoid with softmax: P(Y=k|x) = e^(x'β□) / Σ□ e^(x'β□). Each class gets its own weight vector β□. The softmax ensures all class probabilities sum to 1. Training uses categorical cross-entropy loss.

The Perceptron

The perceptron is the simplest possible linear classifier: $\hat{y} = \text{sign}(\theta \cdot x + \theta_0)$. Unlike logistic regression, it produces hard predictions (no probabilities). The update rule is simple: whenever a point is misclassified, nudge the weight vector toward correctly classifying it.

- Perceptron convergence theorem: if the data is linearly separable, the perceptron WILL converge. If not linearly separable, it oscillates forever.
- The number of updates needed is at most $(R/\gamma)^2$ where R is the radius of the data and γ is the margin (distance between classes).

The perceptron is historically important — it's the conceptual ancestor of neural networks. Each neuron in a neural network is essentially a perceptron with a smooth activation function instead of a hard sign function.

Bias-Variance for Logistic Regression

Since logistic regression produces a linear decision boundary, it has high bias if the true boundary is curved. However, you can add polynomial/interaction features to reduce this bias (at the cost of variance). The regularization parameter C = 1/λ controls the tradeoff: small C means strong regularization (higher bias, lower variance).

---

6. Performance Evaluation

The Confusion Matrix

Every classification result falls into one of four bins. Understanding these is fundamental:

	Predicted Positive	Predicted Negative
Actual Positive	TP (True Positive)	FN (False Negative) — 'Miss'
Actual Negative	FP (False Positive) — 'False alarm'	TN (True Negative)

Key Metrics — When to Use Each

Metric	Formula	Use When
Accuracy	(TP+TN) / N	Balanced classes, costs are equal
Precision	TP / (TP+FP)	False alarms are costly (spam filter: don't delete real emails)
Recall (TPR/Sensitivity)	TP / (TP+FN)	Missing positives is costly (cancer screening: don't miss cancer)
F1 Score	2·Prec·Rec / (Prec+Rec)	When you need balance between precision and recall
Fβ Score	(1+β²)·Prec·Rec / (β²·Prec+Rec)	β>1: recall matters more; β<1: precision matters more
Specificity (TNR)	TN / (TN+FP)	How well the model identifies negatives
FPR	FP / (FP+TN)	False alarm rate — used in ROC curves

ROC Curves and AUC

The ROC (Receiver Operating Characteristic) curve plots TPR (recall) vs. FPR as you vary the classification threshold τ from 0 to 1. At τ=0, everything is predicted positive (TPR=1, FPR=1). At τ=1, nothing is positive (TPR=0, FPR=0).

AUC (Area Under the Curve) summarizes the whole curve in one number: AUC=1.0 is perfect; AUC=0.5 is random guessing (diagonal line). AUC answers: 'What's the probability that a randomly chosen positive example scores higher than a randomly chosen negative example?'

> *Threshold selection: raising τ → fewer predicted positives → precision ↑, recall ↓. Lowering τ → more predicted positives → precision ↓, recall ↑. The threshold is a business/ethical decision, not a purely technical one.*

Precision-Recall Curves and Imbalanced Data

When classes are highly imbalanced (e.g., 1% fraud), accuracy is useless — predicting 'not fraud' always gives 99% accuracy! The ROC curve can also be misleading because FPR looks small even when FP count is large relative to true positives.

In these cases, use Precision-Recall (PR) curves instead. PR curves show the tradeoff between precision and recall without involving TN, which is huge and inflates ROC metrics. PR-AUC gives a more honest picture of performance on the minority class.

Regression Metrics

- MSE (Mean Squared Error): $\Sigma(y_i - \hat{y}_i)^2/n$. Penalizes large errors heavily (squaring). Use when big errors are especially bad.
- MAE (Mean Absolute Error): $\Sigma|y_i - \hat{y}_i|/n$. More robust to outliers. Use when outliers are legitimate data points you don't want to over-penalize.
- R^2: 1 - SS_res/SS_tot. Proportion of variance explained. Unitless and interpretable, but can be misleading — always look at absolute error too.
- Huber loss: behaves like MSE for small errors, MAE for large ones. Best of both worlds for noisy data.

Handling Class Imbalance

- SMOTE: creates synthetic minority class samples by interpolating between existing minority points
- Undersampling: randomly remove majority class samples (loses data)
- Adjust threshold τ: lower it to increase recall on minority class
- Class weights: penalize misclassification of minority class more heavily during training
- Metric choice: use F1, PR-AUC, or Fβ instead of accuracy

---

7. Model Selection & Cross-Validation

The Train/Val/Test Split

This is one of the most important concepts in applied ML, and violations are extremely common in practice:

- Training set: fit model parameters (e.g., weights, coefficients)
- Validation set: tune hyperparameters (e.g., regularization strength, k in KNN, tree depth)
- Test set: final, honest evaluation — used EXACTLY ONCE at the very end

> *Never tune hyperparameters on the test set! If you try 50 different hyperparameter values and pick the one with the best test performance, you've effectively trained on the test set. Your reported error will be optimistic.*

K-Fold Cross-Validation

Split data into k equal folds. For each fold: train on the other k-1 folds, evaluate on the held-out fold. Average the k evaluation scores. This gives a more reliable estimate of generalization performance than a single train/val split.

Standard choices: k=5 or k=10. These give a good bias-variance tradeoff in the CV estimate itself. LOOCV (k=n) is nearly unbiased but has high variance (each model is trained on nearly the same data) and is computationally expensive.

> *Stratified CV: for classification, ensure each fold has the same class proportions as the original dataset. Critical for imbalanced data — without it, some folds might have no minority class examples.*

Data Leakage — The Silent Killer

Data leakage occurs when information from the test/validation set 'leaks' into the training process, giving you an optimistic estimate of performance that won't hold in deployment.

The most common mistake: fitting preprocessing (scaling, imputation, feature selection) on the FULL dataset, then splitting. Even though you didn't look at the test labels, information about the test distribution has influenced your preprocessing.

> *Rule: ALL preprocessing must be fit on the training fold only, then applied to validation/test. Use sklearn Pipelines — they enforce this correctly.*

Other leakage examples: using future data in time-series (predicting next week's sales using next month's weather), including features derived from the target (predicting whether a patient was admitted using their discharge diagnosis).

Hyperparameter Search Strategies

- Grid search: exhaustively tries all combinations from a predefined grid. Fine-grained but exponentially expensive in high dimensions.
- Random search: randomly samples from the hyperparameter space. Surprisingly effective — often finds good solutions faster because it covers more of each dimension independently.
- Bayesian optimization: builds a probabilistic model of the objective function and picks next configurations intelligently. Best for expensive evaluations.
- The 1-SE rule: if a simpler model is within 1 standard error of the best CV score, prefer the simpler model. This builds in a parsimony preference and accounts for the uncertainty in our CV estimate.

---

8. Decision Theory

Loss Functions and Risk

Not all errors cost the same. Diagnosing a sick patient as healthy (false negative) may cost far more than diagnosing a healthy patient as sick (false positive). Decision theory formalizes this.

The loss function $\lambda(\alpha_i|\omega_j)$ specifies the cost of predicting class i when the truth is class j. The expected risk of predicting class i for a given x is $R(\alpha_i|x) = \sum_j \lambda(\alpha_i|\omega_j)P(\omega_j|x)$. We choose the action that minimizes this risk.

Bayes Optimal Classifier

Under 0-1 loss (all errors cost the same), minimizing risk is equivalent to predicting the most probable class: $f^*(x) = \text{argmax}_k P(Y=k|x)$. This gives the Bayes error rate R^* — the minimum achievable error for any classifier on this distribution. No model can do better than R^* in expectation.

Optimal Threshold from Costs

Given cost cFP (cost of a false positive) and cFN (cost of a false negative), the optimal threshold is:

$$\tau^* = cFP / (cFP + cFN)$$

Examples:

- Cancer screening: missing cancer (FN) is far worse than a false alarm (FP). So cFN >> cFP, making τ^* very small — you cast a wide net and catch more positives, accepting more false alarms.
- Spam filter: sending a real email to spam (FN for spam detection, FP for real email) is annoying. So you might set a higher threshold — only mark as spam if you're very confident.
- Fraud detection: depends on the cost of investigating false alarms vs. the cost of missed fraud.

Neyman-Pearson Criterion

A constrained version: maximize TPR subject to FPR ≤ α (a hard budget on false alarms). This is common in security/medical contexts where you have a strict limit on how often you can be wrong in one direction. It gives a specific operating point on the ROC curve.

---

9. Regularization

Why Regularize?

Regularization deliberately adds a penalty to the loss function to discourage overly large or complex models. The goal: accept some bias in exchange for a large reduction in variance, leading to better test performance even if training performance worsens.

> *Core principle: regularization trades bias for variance. The optimal λ balances these — use cross-validation to find it.*

Ridge Regression (L2)

Ridge adds a penalty proportional to the sum of squared coefficients:

$$\hat{\beta}_{ridge} = \text{argmin } ||y - X\beta||^2 + \lambda||\beta||^2$$

The closed-form solution is $\hat{\beta} = (X'X + \lambda I)^{-1}X'y$. Note the $+\lambda I$: this always makes the matrix invertible, even when $X'X$ is singular due to multicollinearity or $p > n$!

Effect: all coefficients are shrunk toward zero, but none become exactly zero. Ridge distributes the 'blame' across correlated features rather than picking one arbitrarily.

- Use when: many features each contribute a small amount, features are correlated, you want stable predictions
- Connection to MAP: Ridge ≡ MAP with Gaussian prior $p(\beta) = N(0, \sigma^2/\lambda \cdot I)$

LASSO (L1)

LASSO uses the L1 norm penalty:

$$\hat{\beta}_{LASSO} = \text{argmin } ||y - X\beta||^2 + \lambda\Sigma|\beta_i|$$

The geometric reason for sparsity: the L1 penalty's constraint region is a diamond/hypercube shape with corners at the axes. The solution tends to land exactly on a corner, which corresponds to most coefficients being exactly zero.

This gives automatic feature selection: LASSO 'turns off' irrelevant features completely. You get a sparse, interpretable model.

- No closed form — must use coordinate descent or subgradient methods
- When features are correlated, LASSO arbitrarily picks one — can be unstable
- Use when: you believe only a few features matter (sparse truth), you want automatic feature selection
- Connection to MAP: LASSO ≡ MAP with Laplace prior $p(\beta) = (\lambda/2)\exp(-\lambda|\beta|)$

Elastic Net

Combines Ridge and LASSO: $\lambda_1||\beta||_1 + \lambda_2||\beta||_2^2$. Gets sparsity from LASSO and the grouping effect from Ridge (correlated features are selected/dropped together rather than arbitrarily). Best of both worlds for high-dimensional data with correlated features.

Other Regularization Methods

- Early stopping: stop gradient descent before it converges. Equivalent to implicit L2 regularization — the longer you train, the more the model fits noise.
- Dropout (neural nets): randomly set a fraction p of activations to zero during each training step. Forces the network to learn redundant representations — no single neuron can be relied upon. At test time, scale activations by (1-p).
- Data augmentation: create modified copies of training data (flip images, add noise, crop). Effectively increases dataset size and forces invariance.
- AIC: $2k - 2\ell$ (k=parameters, ℓ=log-likelihood). BIC: $k \cdot \ln(n) - 2\ell$. Both penalize complexity; BIC penalizes more strongly as n grows. Use for comparing models with different numbers of parameters.

Choosing λ

Search on a log scale (e.g., 10^{-4} to 10^2) — regularization effects often span orders of magnitude. Use k-fold CV to find the λ that minimizes validation error. The 1-SE rule suggests picking the most regularized (simplest) model within 1 SE of the best performance.

10. Generative vs. Discriminative Models

The Fundamental Distinction

	Discriminative	Generative
What they model	P(Y|X) directly	P(X|Y) and P(Y) separately
How they classify	Direct boundary	Via Bayes: P(Y|X) ∝ P(X|Y)P(Y)
Assumptions	Fewer	More (about data distribution)
With lots of data	Usually better	Competitive
With little data	Can overfit	Often better (uses prior structure)
Can generate samples?	No	Yes
Examples	Logistic regression, SVM, NN	Naive Bayes, LDA, GMM

Naive Bayes

Naive Bayes is a generative classifier that makes one strong assumption: all features are conditionally independent given the class. Despite being 'naive' (this assumption is almost always violated in practice), it works surprisingly well, especially for text.

> *$\hat{y} = \text{argmax}_k [\log P(Y=k) + \Sigma_j \log P(x_j|Y=k)]$*

The log form is key for computation — we add log-probabilities instead of multiplying small probabilities (which would underflow to zero).

Variants for different data types:

- Gaussian NB: assumes $P(x_j|Y=k) = N(\mu_{jk}, \sigma^2_{jk})$. For continuous features.
- Bernoulli NB: features are binary (word present/absent). Good for short text.
- Multinomial NB: features are counts (word frequencies). Standard for text classification.

Laplace smoothing: add α (typically 1) to all counts before normalizing. Prevents zero probabilities when a word didn't appear in training for a particular class. Higher α = more smoothing = higher bias but avoids zero-probability catastrophes.

LDA — Linear Discriminant Analysis

LDA assumes each class has a Gaussian distribution with class-specific mean μ_k but SHARED covariance matrix Σ. This shared covariance assumption is what forces the decision boundaries to be linear.

The discriminant function: $\delta_k(x) = \mu_k'\Sigma^{-1}x - (1/2)\mu_k'\Sigma^{-1}\mu_k + \log \pi_k$. Predict the class with the highest $\delta_k(x)$.

LDA is also a dimensionality reduction technique! By projecting onto the discriminant axes (the directions that best separate classes), you can reduce dimensions while preserving class-discriminative information. Used as a preprocessing step before other classifiers.

QDA — Quadratic Discriminant Analysis

QDA relaxes the shared covariance assumption — each class gets its own Σ_k. This creates quadratic (curved) decision boundaries. More flexible than LDA but requires estimating p(p+1)/2 parameters per class, which is problematic when n is small relative to p.

> *Rule of thumb: prefer LDA when n is small relative to p (fewer parameters), or when the equal-covariance assumption seems reasonable. Use QDA when you have enough data and covariances clearly differ between classes.*

11. Trees & Ensembles

Decision Trees

A decision tree recursively partitions the feature space using axis-aligned splits. At each node, it asks 'feature $x_j \leq$ threshold?' and sends data left or right. Each leaf contains a prediction (the mean for regression, most common class for classification).

Impurity Measures

- Gini impurity: $1 - \sum_k p_k^2$. Measures probability of misclassifying a randomly selected item. At 0 when the node is pure; maximum at 0.5 for binary.
- Entropy: $H = -\sum_k p_k \log_2(p_k)$. Information-theoretic measure of disorder. Also 0 when pure. Slightly more computationally expensive than Gini.
- Information gain = H(parent) - $\sum_c (n_c/n) H(c)$. How much entropy is reduced by the split?
- For regression: minimize variance (RSS) within each child node.

Bias-Variance for Trees

A fully grown tree (no regularization) has near-zero training error — it essentially memorizes the training data. But it has very high variance. Regularization strategies:

- Max depth: limit how deep the tree can grow
- Min samples per leaf: require a minimum number of samples before splitting
- Pruning (cost-complexity pruning): grow a full tree, then prune back using a penalty α on the number of leaves. Higher $\alpha \rightarrow$ more pruning $\rightarrow$ simpler tree.

Bagging (Bootstrap Aggregating)

Bagging is a variance reduction technique: train B models on different bootstrap samples (sampling with replacement from the training data), then average predictions. The key insight is that averaging uncorrelated predictions reduces variance proportionally to 1/B.

OOB (Out-Of-Bag) validation: each bootstrap sample excludes about 37% of the data. These excluded samples provide a free validation estimate without needing a separate validation set.

Random Forests

Random Forest = Bagging + random feature subsets. At each split, only a random subset of m features is considered ($m \approx \sqrt{p}$ for classification, p/3 for regression). This decorrelates the trees further — even if one feature is very strong, it won't dominate every tree.

The benefit: correlated trees (all looking at the same strong feature) give less variance reduction when averaged. By randomizing features, Random Forest creates more diverse trees, and their average has lower variance.

> *Adding more trees never hurts performance (variance can only decrease or stay the same), but there are diminishing returns and it gets slower. 100-500 trees is usually sufficient.*

Feature importance: measured as the average decrease in impurity (Gini/entropy) across all splits on that feature, weighted by the number of samples. Useful for feature selection, though correlated features split importance between them.

Boosting — AdaBoost

Boosting takes a different approach: build trees sequentially, with each tree focusing on the mistakes of the previous ones. After each round, upweight the misclassified examples so the next tree pays more attention to them.

Learner weight $\alpha_t = (1/2)\ln((1-\varepsilon_t)/\varepsilon_t)$ where ε_t is the weighted error. A better-performing tree gets a larger vote. Final prediction: $H(x) = \text{sign}(\sum_t \alpha_t h_t(x))$.

Boosting primarily reduces bias (each tree corrects previous mistakes) rather than variance. This is the opposite emphasis from bagging/random forests.

Gradient Boosting

Gradient Boosting generalizes AdaBoost. At each step, fit a new tree to the negative gradient of the loss function — the 'pseudo-residuals.' This works for any differentiable loss, not just classification. For MSE loss, the pseudo-residuals are simply the residuals $y_i - \hat{y}_i$.

Intuition: you're doing gradient descent in function space. Each new tree is a step in the direction that most reduces the loss.

XGBoost — Extreme Gradient Boosting

XGBoost improves on standard gradient boosting with: L2 leaf regularization (penalizes large leaf values), second-order Taylor approximation of the loss (better optimization), column and row subsampling (like random forests, reduces overfitting), and efficient implementation with parallelization.

> *XGBoost is the Kaggle workhorse for tabular data. It's often the first thing to try after a baseline linear model. Key hyperparameters: n_estimators (how many trees), learning_rate (shrinkage per tree), max_depth (tree complexity), subsample/colsample_bytree (regularization).*

Unlike Random Forest, boosting IS sensitive to noise because each tree tries to fit the residuals — including noisy residuals. Use early stopping to prevent overfitting.

Tree vs. Forest vs. Boosting — When to Use

Method	Primary Reduction	Sensitive to Noise?	Interpretable?	Best For
Single Tree	—	Yes (overfits)	Yes	Baseline, understanding
Random Forest	Variance	Somewhat	Feature importance only	Robust tabular baseline
Gradient Boosting/XGBoost	Bias (then variance)	More sensitive	Feature importance only	Competitive tabular performance

---

12. Neural Networks

Architecture and Forward Pass

A neural network is a sequence of transformations: $z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}$ (linear transformation), then $a^{(l)} = f(z^{(l)})$ (nonlinear activation). By stacking multiple layers with nonlinear activations, the network can approximate any continuous function (Universal Approximation Theorem).

Activation Functions — Why They Matter

- Sigmoid: output in (0,1). Problem: vanishing gradient — for large $|x|$, $\sigma'(x) \approx 0$, so gradients die in deep networks. Use only at output for binary classification.
- Tanh: output in (-1,1). Centered (better than sigmoid), but still has vanishing gradient problem.
- ReLU ($\max(0,x)$): the default hidden-layer activation. Gradient is 1 for positive inputs (no vanishing gradient). Fast to compute. Problem: 'dead neurons' — if a neuron gets stuck in the negative region, its gradient is always 0 and it never updates.
- Leaky ReLU ($\max(\alpha x, x)$ with small α like 0.01): fixes dead neuron problem by allowing a small gradient when $x<0$.
- Softmax at output for multiclass: ensures outputs sum to 1, interpretable as probabilities.

Loss Functions

- Regression: MSE (penalizes large errors) or MAE (robust)
- Binary classification: Binary Cross-Entropy (BCE) = $-(y \cdot \log(\hat{p}) + (1-y) \cdot \log(1-\hat{p}))$
- Multiclass: Categorical Cross-Entropy (CCE) = $-\Sigma\square\ y\square\ \log(\hat{p}\square)$

Backpropagation

Backpropagation efficiently computes gradients of the loss with respect to all parameters by applying the chain rule backwards through the network. The error signal $\delta^{(l)} = (W^{(l+1)})'\delta^{(l+1)} \odot f'(z^{(l)})$ flows backwards, and the weight gradient $\partial L/\partial W^{(l)} = \delta^{(l+1)}(a^{(l)})'$ is computed at each layer.

> *Conceptually: backprop answers 'how much does each weight contribute to the final error?' It's just the chain rule applied systematically. The key insight is computing intermediate values (δs) once and reusing them.*

Optimizers

- SGD: simple gradient descent on mini-batches. Works but can be slow and get stuck in saddle points.
- Momentum: maintains a running average of past gradients (velocity) to accelerate through flat regions and dampen oscillations. Think of a ball rolling downhill — it doesn't stop immediately at the bottom.
- Adam: adaptive learning rates per parameter, using estimates of both the gradient (first moment) and squared gradient (second moment). Usually the best default for neural networks.

Initialization — Xavier and He

Poor initialization can cause vanishing or exploding gradients before training even begins.

- Xavier initialization: $W \sim N(0, 2/(n_{in}+n_{out}))$. Designed for sigmoid/tanh — keeps activation variance roughly constant across layers.
- He initialization: $W \sim N(0, 2/n_{in})$. Designed for ReLU — accounts for the fact that ReLU kills half its inputs on average.

Batch Normalization

Batch norm normalizes the activations of each layer across the mini-batch (subtract mean, divide by std), then applies learned scale γ and shift β parameters. Benefits: faster training, allows higher learning rates, mild regularization effect (the noise from mini-batch statistics acts like a regularizer). Nearly universally used in modern deep learning.

Hyperparameters for Neural Networks

- Learning rate: the most critical HP. Too high → diverge. Too low → slow. Use LR warmup (ramp up from small LR) and scheduling (reduce over time).
- Batch size: larger batches give smoother gradient estimates but can generalize worse (less noise). 32-256 is standard. Very large batches may need different LR.
- Depth/width: more depth/width → more capacity → lower bias but higher variance. Regularize accordingly.
- Dropout rate p: set fraction p of activations to 0 during training. Typical: 0.1-0.5. Doesn't hurt bias much but significantly reduces overfitting.
- Early stopping: monitor validation loss; stop when it stops improving. The patience parameter controls how many epochs to wait before stopping.

---

13. CNNs & Transformers

Convolutional Neural Networks (CNNs)

CNNs exploit the structure of image data: nearby pixels are related, and the same patterns (edges, corners, textures) appear at different locations. CNNs achieve this through two key mechanisms:

- Local connectivity: each filter looks at a small patch (e.g., 3×3) of the input, not the entire image. Drastically reduces parameters.
- Weight sharing: the same filter is applied across all positions. This gives translation equivariance — the same edge detector works everywhere in the image.

Output size formula: ⌊(n + 2p - f)/s⌋ + 1, where n=input size, p=padding, f=filter size, s=stride. Standard: f=3, p=1, s=1 preserves spatial dimensions.

Max pooling (typically 2×2): takes the maximum in each region, downsampling by 2×. Adds translation invariance (small shifts don't change the output) and reduces computation.

Typical architecture: Conv → BN → ReLU → Pool → repeat → Flatten → FC → Softmax. Deep features represent increasingly abstract concepts: early layers detect edges, later layers detect shapes, faces, objects.

Transformers — Self-Attention

Transformers revolutionized NLP and are now used for images, audio, and beyond. The key mechanism is self-attention: every position in the sequence can attend to (look at and incorporate information from) every other position.

Q, K, V matrices: for each input, compute a Query (what am I looking for?), Key (what do I offer?), and Value (what information do I carry?). Attention weights: softmax(QK' / √d□), applied to V.

> *Why divide by √d□? For large d□, the dot products QK' grow large, pushing softmax into regions with near-zero gradients (saturation). Scaling prevents this.*

Multi-head attention: run h parallel attention heads, each learning different relationships (some may focus on syntax, others on semantics). Concatenate and project.

Positional encoding: unlike RNNs, transformers process all positions simultaneously (parallelizable!), so they need explicit position information added to the embeddings.

Architectures:

- Encoder-only (BERT): good for understanding tasks — classification, named entity recognition
- Decoder-only (GPT): good for generation — text completion, language modeling
- Encoder-Decoder (T5, original Transformer): good for sequence-to-sequence — translation, summarization

---

14. PCA & Dimensionality Reduction

The Curse of Dimensionality

As the number of dimensions d grows, something counterintuitive happens: data becomes exponentially sparse. To maintain the same density, you need n ~ e^d samples. In 100 dimensions, no amount of reasonable data can densely fill the space.

Also: distances concentrate. The ratio max_distance/min_distance → 1 as d → ∞. This means 'nearest neighbors' aren't very near — KNN degrades. Density estimation becomes unreliable. Many algorithms break down.

> *This is why dimensionality reduction is not just a nicety — it's often necessary for many ML algorithms to work well.*

PCA — Principal Component Analysis

PCA finds the orthogonal directions of maximum variance in the data. The first PC points in the direction of greatest variance, the second PC (orthogonal to the first) captures the next most variance, and so on.

Steps:

- 1. Center the data: $\tilde{X} = X - \bar{x}$ (critical — PCA without centering gives nonsense)
- 2. Compute covariance matrix $S = (1/n-1)\tilde{X}'\tilde{X}$ or SVD: $\tilde{X} = U\Sigma V'$
- 3. Sort eigenvectors by eigenvalue (variance captured) in descending order
- 4. Project: $Z = \tilde{X} \cdot V[:,1:d']$ — the d' columns of V are the principal components

How many components to keep? Use a scree plot (plot eigenvalues in order — look for the 'elbow') or choose d' to retain ≥ 90% of total variance. The proportion of variance explained by PC k is $\lambda_k / \Sigma\lambda$□.

> *If features have different units (e.g., one in kilograms, another in dollars), standardize (z-score) before PCA. Otherwise, the high-variance feature will dominate all principal components.*

When PCA Helps

- Removes correlated/redundant features — many features capture the same underlying variation
- Noise reduction — small eigenvalues often correspond to noise dimensions
- Visualization — project to 2D or 3D to see cluster structure
- Preprocessing before linear models when p >> n

> *PCA is unsupervised — it finds directions of high variance without using labels. This may not align with discriminative directions. LDA is the supervised alternative.*

Nonlinear Alternatives

- t-SNE: maps high-dimensional data to 2D/3D preserving local structure (nearby points stay nearby). Excellent for visualization. NOT for general dimensionality reduction or predictive use — the axes have no interpretation and the method is stochastic.
- UMAP: similar to t-SNE but preserves global structure better, faster, and can be applied to new data points. Increasingly popular.
- Kernel PCA: applies PCA in a kernel-induced feature space, capturing nonlinear structure. Computationally expensive.

---

15. Clustering (Unsupervised Learning)

K-Means

K-Means iteratively assigns points to clusters and updates cluster centers. It minimizes within-cluster sum of squares (inertia): $\Sigma_k \Sigma_{i \in C_k} ||x_i - \mu_k||^2$.

Algorithm:

- 1. Initialize k centroids (randomly or K-means++)
- 2. E-step: assign each point to nearest centroid
- 3. M-step: recompute each centroid as the mean of its assigned points
- 4. Repeat until assignments stop changing

Important limitations:

- Assumes spherical, equal-size clusters. Fails badly on elongated or non-convex clusters.
- Sensitive to initialization — can converge to different local minima. Use K-means++ (smarter initialization) and multiple restarts (n_init parameter).
- Must specify k in advance — use elbow method or silhouette score to choose k.

> *K-means++ initialization: place first centroid randomly, then each subsequent centroid is chosen with probability proportional to d^2 (distance squared) from nearest existing centroid. This spreads initial centroids apart and avoids bad runs.*

Silhouette Score

Silhouette score $s(i) = (b(i) - a(i)) / \max(a(i), b(i)) \in [-1, 1]$, where a(i) = average distance to points in same cluster, b(i) = average distance to points in nearest other cluster. Near 1 = well-clustered; near 0 = on boundary; negative = possibly in wrong cluster. Average over all points gives overall cluster quality.

GMM — Gaussian Mixture Models

GMM is a probabilistic, 'soft' version of K-means. Instead of hard cluster assignments, each point has a probability of belonging to each Gaussian component. The model: $p(x) = \Sigma_k \pi_k N(x|\mu_k, \Sigma_k)$.

EM algorithm:

- E-step: compute $r_{ik} = P(Z_i=k|x_i)$ — the responsibility of cluster k for point i
- M-step: update π_k, μ_k, Σ_k using the responsibilities as soft weights

Advantages over K-means: handles elliptical clusters (full covariance Σ_k), provides soft memberships, can do density estimation.

- Covariance types: full (most flexible, most params), diagonal (assumes independent features), spherical (= K-means equivalent, fewest params)

Hierarchical Clustering

Starts with n singleton clusters and greedily merges the closest pair at each step. Produces a dendrogram — a tree showing the hierarchy of merges. You can 'cut' this tree at any height to get any number of clusters.

Linkage criteria (how to measure distance between clusters):

- Single linkage: distance = minimum distance between any two points in the clusters. Prone to chaining — long, stringy clusters.
- Complete linkage: maximum distance. Creates compact, spherical-ish clusters.
- Average linkage: average of all pairwise distances. A compromise.
- Ward's method: merge clusters that minimize the increase in total within-cluster variance. Usually produces the most useful clusters — the default choice.

> *Hierarchical clustering is especially useful when you don't know k in advance, or when the hierarchical structure itself is meaningful (e.g., species taxonomy, document organization).*

DBSCAN — Density-Based Clustering

DBSCAN defines clusters as dense regions separated by sparse regions. Points are classified as:

- Core point: has ≥ MinPts neighbors within radius ε
- Border point: within ε of a core point but not a core point itself
- Noise/outlier: neither — not within ε of any core point

Clusters are connected components of core points (plus their border points). DBSCAN does NOT require specifying k and naturally identifies outliers. It can find arbitrarily shaped clusters.

Parameter selection: ε controls the neighborhood radius (larger → larger clusters, fewer outliers); MinPts typically ≈ 2d where d is the number of features. Plot sorted k-nearest-neighbor distances to find a good ε at the 'elbow.'

Limitation: struggles with varying density — one ε/MinPts setting can't capture both dense and sparse clusters simultaneously. HDBSCAN (hierarchical DBSCAN) solves this.

---

16. Reinforcement Learning

The Framework

RL is fundamentally different from supervised learning: there's no labeled dataset. An agent learns by interacting with an environment, taking actions, and receiving rewards. The goal is to learn a policy — a mapping from states to actions — that maximizes cumulative reward.

Key building blocks:

- State s: description of the current situation
- Action a: choice made by the agent
- Reward r: scalar feedback signal from the environment
- Policy π(a|s): probability distribution over actions given state
- Transition function P(s'|s,a): probability of next state (may be unknown)

> *The Markov property: the next state depends only on the current state and action, not on history. P(s_{t+1}|s_t) = P(s_{t+1}|s_t, s_{t-1},...). This is what makes the math tractable.*

Return and Discount Factor

The return $G_t = \Sigma_t \gamma^k R_{t+k+1}$ sums up future rewards. The discount factor $\gamma \in [0,1]$:

- $\gamma \to 0$: myopic agent that only cares about immediate reward
- $\gamma \to 1$: far-sighted agent that weights future rewards nearly equally to immediate ones
- $\gamma < 1$ is needed for continuing tasks (infinite horizon) to ensure the sum converges

Value Functions

Value functions estimate 'how good' it is to be in a given state or take a given action:

- V^π(s): expected return starting from state s and following policy π forever
- Q^π(s,a): expected return starting from state s, taking action a, then following π

Relationship: V^π(s) = Σ_a π(a|s)Q^π(s,a). If you know Q, you can recover V (and derive the optimal policy without needing the transition model).

Bellman Equations — The Key Recursion

The Bellman equations express value functions recursively: the value of a state = immediate reward + discounted value of next state. This 'backup' property is what makes DP and TD learning work.

> *V^π(s) = E_π[R_{t+1} + γV^π(S_{t+1}) | S□=s]*

Bellman optimality equations: substitute max over actions for the expectation over π. V*(s) = max_a Σ P(s',r|s,a)[r + γV*(s')]. Finding V* gives us the optimal policy for free: π*(s) = argmax_a Q*(s,a).

Dynamic Programming (Requires Known MDP)

DP algorithms solve the Bellman equations when we have full knowledge of P and R:

- Policy Evaluation: iterate Bellman expectation equations to get V^π (how good is this policy?)
- Policy Improvement: given V^π, act greedily to get a better policy π' (guaranteed V^{π'} ≥ V^π)
- Policy Iteration: alternate evaluation and improvement until stable. Convergence guaranteed.
- Value Iteration: do only 1 sweep of evaluation per improvement step. Converges to V* faster.

Monte Carlo Methods — Model-Free, Episode-Based

MC doesn't need to know P or R. It learns from experience: run complete episodes, then average the actual returns G□ observed. No bootstrapping — uses real returns, not estimated future values.

- Advantage: zero bias (uses actual returns, not estimates)
- Disadvantage: high variance (single episode is noisy), only works for episodic tasks

> *First-visit MC: only count the first time a state is visited within an episode. Every-visit MC: count every visit. First-visit is unbiased; every-visit has lower variance.*

Temporal Difference Learning — Model-Free, Online

TD combines the model-free aspect of MC with the online (step-by-step) aspect of DP. The key innovation: update using the TD target r + γV(s') instead of waiting for the actual return.

> *V(s) ← V(s) + α[r + γV(s') - V(s)] (TD error δ□ = r + γV(s') - V(s))*

- Biased: uses V(s') as an estimate, not the true return (bootstrapping)
- Lower variance than MC: smoother updates, doesn't wait for full episodes
- Online: can learn during an episode, crucial for continuing tasks

Q-Learning and SARSA

Q-Learning directly learns the optimal Q* regardless of which policy is being followed (off-policy):

> *Q(s,a) ← Q(s,a) + α[r + γ max_a' Q(s',a') - Q(s,a)]*

SARSA uses the action actually taken under the current policy (on-policy):

> *Q(s,a) ← Q(s,a) + α[r + γQ(s',a') - Q(s,a)], where a' ~ π*

- Q-learning: converges to Q* regardless of behavior, but can learn overly optimistic values if the behavior policy is dangerous
- SARSA: learns the value of the actual policy being followed, safer in environments with real costs (cliff-walking problem)

Exploration vs. Exploitation

The fundamental dilemma: exploit what you know (take the best known action) or explore (try new actions that might be better). Too much exploitation → stuck in local optima. Too much exploration → never capitalizes on knowledge.

- ε-greedy: with probability ε take a random action; otherwise take argmax Q. Decay ε over time (explore early, exploit later).
- UCB (Upper Confidence Bound): $a^* = \text{argmax}_a[Q(a) + c\sqrt{(\ln t / N(a))}]$. The second term is an 'optimism bonus' for rarely tried actions. More principled than ε-greedy — automatically reduces exploration of well-understood actions.

Deep RL — DQN

When the state space is large (images, continuous states), we can't store a Q-table. DQN uses a neural network $Q(s,a;\theta)$ to approximate Q^*. Two critical tricks for stability:

- Experience replay: store transitions (s,a,r,s') in a buffer; sample random mini-batches to train. Breaks temporal correlations between consecutive samples that would destabilize training.
- Target network: maintain a separate, slowly-updated copy of the Q-network as the target. Prevents the target from moving every step, which creates a 'chasing your own tail' problem.

---

17. Gradient Descent & Optimization

The Core Algorithm

$\theta \leftarrow \theta - \eta \nabla_\theta L$. The learning rate η is the most important hyperparameter. Too high: oscillates, may diverge. Too small: takes forever, may get stuck. Good ranges: 10^{-4} to 10^{-1}, but problem-dependent.

Variants

- Batch GD: compute gradient on ALL training data before updating. Stable, but extremely slow for large datasets. Gradient is exact.
- Stochastic GD (SGD): update after EACH sample. Fast (one gradient per update), noisy (single-sample gradient is a rough estimate), but noise can help escape local minima.
- Mini-batch GD: update after each batch of 32-256 samples. Standard practice — balances speed and stability.

Non-Convexity in Deep Learning

Neural network loss functions are non-convex — there are many local minima and saddle points. Good news: in high-dimensional spaces, true local minima are rare (most 'valleys' are saddle points). SGD noise helps escape saddle points. Multiple random restarts + early stopping often find good solutions.

For convex losses (linear/logistic regression with convex regularization), gradient descent finds the global minimum.

Learning Rate Schedules

- Step decay: multiply LR by a factor (e.g., 0.1) every N epochs
- Cosine annealing: LR follows a cosine curve from η_{max} to η_{min}. Smooth, effective.
- Warmup: start with tiny LR, ramp up to target LR over first few epochs/steps. Critical for Transformers — prevents early instability.

---

18. Practical Reference & Exam Quick Tips

Feature Scaling — When Required vs. Not

Needs Scaling	Does NOT Need Scaling
KNN (distance-based)	Decision Trees
PCA (variance-based)	Random Forest
Neural Networks (GD-based)	Gradient Boosting / XGBoost
Logistic/Linear Regression with GD	Naive Bayes
SVM	Any tree-based method
Ridge / LASSO	

Model Selection Guide

Situation	Try First	Also Consider
Continuous output (regression)	Linear Regression / Ridge	Random Forest, XGBoost
Binary classification, linear	Logistic Regression	LDA, SVM
Binary classification, nonlinear	Random Forest	XGBoost, NN
Text/NLP classification	Naive Bayes (baseline)	Logistic Regression, Transformer
Images	CNN	Pretrained ResNet/EfficientNet
Sequences/NLP (modern)	Transformer (BERT/GPT)	LSTM (legacy)
Tabular data, competitive	XGBoost	Random Forest, Neural Net
Small n, Gaussian features	LDA	QDA, Logistic Regression
Unknown k clusters	DBSCAN / Hierarchical	GMM
Known k, spherical clusters	K-Means	GMM

Diagnosing Overfit vs. Underfit

Symptom	Problem	Fixes
Low train error, high test error	Overfitting (high variance)	More data, regularization, dropout, simpler model, early stopping, data augmentation
High train AND test error	Underfitting (high bias)	More complex model, add features, reduce regularization, train longer, more layers/width
High variance in CV scores	Model too sensitive to data	More regularization, more data, simpler model

Loss Function Reference

Task	Loss Function	Notes
Regression	MSE	Penalizes outliers heavily (squared)
Regression (robust)	MAE or Huber	MAE: robust; Huber: MSE near 0, MAE for large errors
Binary classification	Binary Cross-Entropy	Equivalent to log-loss, MLE for Bernoulli
Multiclass classification	Categorical Cross-Entropy	Generalization to K classes
Imbalanced binary	Weighted BCE or Focal Loss	Weight minority class more heavily

Key Hyperparameter Effects Summary

HP Increase	Bias Effect	Variance Effect	Model
λ (regularization)	↑ (more constrained)	↓ (less overfitting)	Ridge, LASSO, any regularized
k in KNN	↑ (over-smoothed)	↓ (more averaging)	KNN
Tree max depth	↓ (fits more complex patterns)	↑ (memorizes more)	Decision Trees
n_estimators (RF/Boosting)	Flat (no effect)	↓ (more averaging/correction)	RF, Boosting
Learning rate (boosting)	↑ (needs more trees)	↓ (gentler steps)	Gradient Boosting
Polynomial degree d	↓ (more expressive)	↑ (more wiggly)	Polynomial regression
Dropout rate p	↑ (slight)	↓ (significant)	Neural Networks
C = 1/λ in logistic	↓ (less constrained)	↑ (more overfit risk)	Logistic Regression

Common Exam Pitfalls

⚠ Pitfalls to avoid on the exam:

- Confusing bias and variance: complex model → low bias, high variance (not the other way around)
- Saying regularization always hurts: it increases bias but decreases variance — often net positive
- Forgetting that k=1 in KNN perfectly fits training data (0 training error) but has high test error
- Applying feature scaling to tree-based methods — not needed and doesn't help
- Computing preprocessing on full dataset before CV split — this is data leakage
- Confusing MLE and MAP: MAP = MLE + prior. When λ=0, MAP reduces to MLE.
- Saying independent implies uncorrelated but not the other way around — it's reversed: independent → uncorrelated, but uncorrelated ≠ independent
- Forgetting that AUC=0.5 means random performance (not 0)
- Using accuracy for imbalanced datasets — use F1 or PR-AUC instead
- Confusing Q-learning (off-policy) with SARSA (on-policy)
- Saying boosting reduces variance like bagging — boosting primarily reduces bias
- Forgetting that LASSO can't handle correlated features well (picks arbitrarily)

CV Discipline Checklist

✓ Before reporting any results, verify all of these:

- All preprocessing (scaling, imputation, feature selection) is fit on training fold only
- Hyperparameters are tuned on validation set, NOT test set
- Test set is used EXACTLY ONCE at the very end
- For classification, use stratified CV if classes are imbalanced
- Report test set performance, not validation performance
- For time-series data, use time-based splits (not random splits that leak future into past)